PROOFFLOW

Proof of Delivery Management System

TECHNICAL DESIGN DOCUMENT

Demonstration solution for digitising POD capture, review and billing readiness in a full-truckload transport operation.

Document detail	Value
Version	1.0
Date	4 November 2026
Project repository	github.com/Srikanthnaidu13/proofflow-pod-management
Live demonstration	https://proofflow-pod-management.vercel.app/
Application type	Next.js web application; browser-local demonstration data
Deployment	Vercel production deployment; Mumbai region configured in vercel.json

Important scope statement

ProofFlow is a functional demonstration, not a production-ready transport platform. It demonstrates the intended workflows using browser IndexedDB and synthetic/sample trip data. Production authentication, shared cloud persistence, secure external uploads, real SMS, and certified Tally integration are not implemented.

Prepared for technical evaluation. All financial values and operating volumes below are based on the supplied assignment scenario and are planning estimates, not audited company results.

1. Executive Summary

ProofFlow addresses delays and losses caused by paper-based Proof of Delivery (POD) handling. Drivers or hired transporters submit delivery documents, operations staff review document quality and key fields, and billing staff use approved POD records to prepare billing. The application brings these steps into one demonstrable workflow.

- Capture POD images from a driver-oriented mobile interface, with image compression, timestamp and best-effort GPS capture.
- Provide an upload flow for hired drivers using a trip-specific link pattern.
- Use Gemini AI to extract and assess selected POD fields, while keeping human review and approval in control.
- Give operations staff a review queue with editable extracted fields, retake handling and approval.
- Give billing staff a verified-POD queue, CSV demonstration export and a Mark Billed action.
- Show missing-POD ageing and freight-at-risk indicators, and provide a read-only client-facing POD view for eligible records.

The implementation prioritises a small, understandable MVP over fleet tracking, route planning or financial operations features that are outside the assignment scope.

2. Business Problem and Opportunity

The assignment describes a full-truckload transporter operating from Chhatrapati Sambhajinagar, Maharashtra, with approximately 140 owned trucks, 200 hired trucks, 210 staff and annual revenue of ■75–90 crore. Signed LR/POD documents may return late, WhatsApp images may be lost, and billing in Tally is delayed until POD is available.

Operating assumption	Value
Trips per month	2,200
Average freight per trip	■32,000
Annualised trips	26,400
Annualised freight value	■84.48 crore
Current POD-to-invoice cycle	18 days
Target POD-to-invoice cycle	4 days
Potential cycle-time reduction	14 days
Estimated annual financing rate	12%
POD lost/disputed assumption	1.5% of annual trips

2.1 Working-capital estimate

Annual freight throughput = 2,200 trips/month × ■32,000/trip × 12 months = ■84.48 crore.

Potential working capital released by reducing the cycle by 14 days = ■84.48 crore × 14 ÷ 365 ≈ ■3.24 crore.

Indicative annual financing-cost benefit = ■3.24 crore × 12% ≈ ■38.9 lakh.

2.2 Lost/disputed POD estimate

Annual trips = 2,200 × 12 = 26,400. At 1.5%, estimated affected trips = 396. Assuming half of average freight is at risk per affected trip: 396 × ■16,000 = ■63.36 lakh.

Indicative combined annual impact

■38.9 lakh financing-cost opportunity + ■63.36 lakh lost/disputed freight exposure ≈ ■1.02 crore per year. This is an indicative scenario estimate; actual realised benefit depends on recovery rates, billing terms, customer disputes and operating adoption.

3. Goals, Users and Scope

3.1 Primary users

User	Needs	ProofFlow support
Driver (owned fleet)	Simple capture flow, clear instructions, low data usage, language support	Large capture controls, image compression, English/Hindi/Marathi prompts, offline queue
Hired driver / transporter	Submit a POD without a full account setup	Trip-specific upload page pattern and mock link generator
Operations / POD reviewer	Identify incomplete or unclear documents and correct extracted details	Review queue, AI checks, editable fields, retake request and approval
Billing executive	Find approved PODs and prepare billing handoff	Verified queue, CSV export and Mark Billed status
Operations head / MD	Understand missing POD exposure and cycle-time bottlenecks	Dashboard, ageing buckets and freight-at-risk indicators
Client / consignee	View approved delivery evidence	Read-only client POD page for eligible records

3.2 Included in the demonstration

- Driver camera/file capture, client-side image compression and best-effort GPS/time metadata.
- Offline capture queue persisted in browser IndexedDB, with local reconnection handling.
- Hired-driver upload page and generated demo links; actual SMS delivery is mocked.
- AI-assisted POD checks and structured result validation, followed by human review.
- Operations review queue, retake flow, billing queue, CSV import/export and missing-POD dashboard.
- Seed/sample trips, client view and responsive layouts.

3.3 Explicitly out of scope

- Live GPS tracking, trip planning, route optimisation, fuel or driver expense management.
- Production-grade identity management, role enforcement and secure expiring external links.
- Shared cloud file storage, multi-device synchronisation or a central production database.
- Real SMS/WhatsApp delivery, payment collection, automatic invoice generation or sending.
- Certified Tally integration or a guaranteed Tally-import file format.

4. Solution Architecture

The deployed application is a Next.js App Router application. UI pages and client components run in the browser. Demonstration trip state and POD image blobs are stored in browser IndexedDB. The AI analysis request is sent to a Next.js API route, which calls the Google Gemini API using a server-side environment variable.

```
Driver / Hired Driver Browser → Next.js UI → Browser IndexedDB (trips + offline POD blobs)
Operations / Billing Browser → Next.js UI → Browser IndexedDB
Review Queue → POST /api/analyze-pod → Gemini API → Structured analysis response
Billing Queue → CSV demonstration export → Manual billing/Tally handoff
```

Layer	Technology / responsibility
Presentation	Next.js App Router, React, TypeScript, responsive CSS, Inter and Noto Sans Devanagari fonts

Layer	Technology / responsibility
Application logic	Client components for capture, review, dashboard, import and billing workflows
Local persistence	IndexedDB database proofflow-data with trips store; separate proofflow-offline database/store for queued POD blobs
AI service	Next.js POST /api/analyze-pod route; @google/genai client; structured POD analysis response
Deployment	Vercel production deployment; vercel.json specifies region bom1 (Mumbai)
Source control	Public GitHub repository; master branch

Because the core records are browser-local, different browsers or devices do not share trip state or uploaded POD files. The production URL serves the application, but it does not turn IndexedDB into a central data store.

5. Data Model

The demonstration uses a TripRecord object in IndexedDB. It is designed around one trip and its current POD processing state. The stored POD image is represented as a Blob in the browser.

Field group	Representative fields	Purpose
Identity / trip	id, client, vehicle, destination, amount, lrNumber, ewayBill, branch	Identify the consignment, parties, vehicle, freight and branch.
Workflow	status, issue, retakeCount, tripCreatedAt, submittedAt, verifiedAt, billedAt	Track lifecycle and calculate metrics only where timestamps exist.
AI / review	aiConfidence, aiAnalysis, consigneeStamp, quantityReceived, shortageDamageRemarks, deliveryDate, signatureStatus	Store AI observations and reviewer-corrected POD fields.
Evidence / metadata	podBlob, podFileName, latitude, longitude, gpsStatus	Keep local image evidence and best-effort submission metadata.

5.1 POD status lifecycle

The shared status vocabulary is: missing → submitted → verified → billed. Exceptions include needs_review and retake_requested. A retake submission returns to submitted for a new review. A reviewer can approve a sufficiently complete POD or request another capture, subject to the configured retake limit.

The implementation uses an IndexedDB helper that reads the existing trip and merges updates, preserving fields not changed by the current action. CSV import inserts new trip IDs and skips duplicates rather than overwriting existing records.

5.2 Sample data

The application includes synthetic sample trip CSV files and seed trips for demonstration. Seed records may not contain creation or submission timestamps; dashboard metrics avoid inventing timestamps and display unavailable values rather than fabricating cycle-time performance.

6. Core Workflows

6.1 Driver POD capture

- Driver opens the driver workspace and selects the relevant trip.
- Driver captures an image or selects a file. The client compresses it to a JPEG target below 300 KB, with a maximum dimension of 1,600 pixels.
- The browser attempts to capture geolocation. If unavailable or denied, the record indicates GPS unavailable rather than inventing coordinates.
- The trip is updated to submitted with the available timestamp, GPS status and local POD Blob.

- When offline, the POD is queued in IndexedDB. Reconnection handling synchronises it into the same browser's local trip store; this is not a cloud upload.

6.2 Hired-driver upload

The application provides /upload-pod?trip= as a demonstration upload page. A dashboard utility generates trip-specific link text. The link is not an authenticated, signed or expiring token, and the page writes to local browser storage only. SMS delivery is mocked.

6.3 AI-assisted review

- Reviewer selects a submitted trip and requests analysis of its POD image.
- The server route validates file type and size, checks GEMINI_API_KEY, and sends the image with a constrained extraction prompt to Gemini.
- The expected structured result covers LR number, consignee stamp, received quantity, shortage/damage remarks, delivery date and signature status, along with confidence and a recommendation.
- The route validates the response structure before returning it to the client.
- Unclear or critical fields remain in manual review. AI does not autonomously approve a POD; a human reviewer must inspect and approve it.
- The reviewer can edit extracted values, request a retake (up to the configured limit) or approve the record.

6.4 Billing and client view

- Billing staff see verified records and records already marked billed.
- CSV export is a demonstration handoff, not a certified Tally format. Mark Billed updates the local trip status and billed timestamp.
- The client POD page is read-only and intended to display only verified/billed records with available image evidence. It relies on the same browser-local data and is not a secure external customer portal.

6.5 Missing-POD monitoring and trip import

The missing-POD dashboard shows counts, freight exposure and ageing buckets based on recorded trip creation timestamps. Trips without a recorded creation timestamp are excluded from ageing calculations. CSV import validates required columns, previews records and skips duplicate IDs.

7. AI Design and Responsible Use

Gemini is used as an assistant for document reading and quality triage, not as the system of record or final approver. The prompt asks the model not to invent missing values, to mark uncertain fields as unclear, and to recommend manual review or retake when evidence is insufficient.

Check	Expected handling
LR number	Extract only when legible; otherwise mark unclear for reviewer correction.
Consignee stamp	Classify visible / not visible / unclear.
Quantity received	Extract a numeric quantity only when supported by the document.
Shortage / damage	Capture remarks and route exceptions to manual review.
Delivery date	Extract a valid date only when readable.
Signature	Classify present / missing / unclear.

Data protection rule for this demo

Use synthetic POD images only. Do not upload real customer, driver or consignee documents to Gemini until the applicable Google service tier, data-handling terms, retention and training settings have been reviewed and approved by the organisation.

The API key is stored as the server-side environment variable GEMINI_API_KEY in Vercel and in the developer's local .env.local file. The key must not be committed to Git or exposed to browser code. The endpoint currently does not implement production authentication, rate limiting, abuse detection or per-customer access controls.

8. UI and Interaction Decisions

- A calm, high-contrast interface with a light neutral background, white surfaces and teal accent supports operational scanning without visual clutter.
- Desktop operations views use a sidebar and content grid; driver capture is designed for a narrow mobile viewport.
- Driver actions use large tap targets, short instructions and English/Hindi/Marathi language options with browser voice prompts.
- Semantic status colours distinguish verified, retake, review and offline states.
- A visible delivery progression communicates Loaded → In transit → Delivered → POD verified → Billed.
- Review and billing pages separate operational decisions from AI suggestions, keeping human approval explicit.

9. Security, Privacy and Operational Limitations

Area	Current demonstration behaviour	Production requirement
Authentication	No production identity provider or enforced login.	Integrate organisation SSO/OIDC; secure sessions and account lifecycle.
Authorisation	Role-oriented UI and workflow concepts; not a security boundary.	Enforce RBAC server-side on every record and file operation.
POD storage	Browser IndexedDB Blob; browser/device-local.	Private object storage, encryption, access policy, retention and backup.
External upload link	Trip ID in query string; no expiry or signature.	Signed, expiring, one-time upload token with abuse controls.
AI endpoint	API key server-side; no production auth/rate limit.	Authenticated API, quotas, rate limits, logging and data minimisation.
Messaging	SMS/link generation is a mock workflow.	Approved SMS provider, consent, delivery status and audit trail.
Billing	CSV demo export and manual Mark Billed action.	Finance-approved Tally integration, reconciliation and audit trail.
Privacy/compliance	Synthetic data guidance; no compliance certification.	Legal/security review, DPDP Act applicability assessment, notices, consent and retention controls.

The demo must not be represented as DPDP-compliant, production-secure or suitable for real customer records. Production deployment requires a threat model, security review, access controls, audit logging, backups, monitoring and incident-response procedures.

10. Verification and Quality Checks

- ESLint: npm run lint — passed in the local development environment.
- TypeScript: npx tsc --noEmit — passed.
- Production build: npm run build — passed.

- The build includes the homepage, dashboard, driver, review, billing, missing-POD, hired-driver upload, client POD and AI API routes.
- Manual UI checks covered route rendering, client portal missing-ID behaviour and sample dashboard/missing-POD data presentation.
- Actual camera/POD image scenarios were intentionally deferred; the demo should be exercised with synthetic images before any live operational pilot.

These checks establish build and static-quality status at the time of preparation. They are not a substitute for end-to-end, cross-browser, security, load or user-acceptance testing.

11. Deployment and Configuration

- Public source repository: https://github.com/Srikanthnaidu13/proofflow-pod-management
- Production URL: https://proofflow-pod-management.vercel.app/
- Hosting: Vercel, Hobby project; vercel.json specifies the Mumbai region identifier bom1.
- Framework: Next.js App Router. Build command: npm run build. Output: Next.js default.
- Required server-side environment variable: GEMINI_API_KEY.
- Local development: npm ci, create .env.local with GEMINI_API_KEY=, then npm run dev.
- Quality checks: npm run lint; npx tsc --noEmit; npm run build.

Never place a real API key in this document, source control, screenshots or public issue reports. The deployed application is a demonstration; its browser-local data is not shared with reviewers using another browser or device.

12. Assumptions and Decisions

- Assignment operating volumes, freight values, POD loss rate and financing rate are scenario inputs supplied for solution design; they are not independently audited.
- The demo uses sample trip data to demonstrate workflows without exposing real customer or driver information.
- GPS is best-effort and depends on device permission, browser support and network/device conditions.
- AI extraction quality depends on image clarity, document layout, handwriting, language and model behaviour; human verification remains mandatory.
- A local IndexedDB prototype was selected to keep the demo deployable without provisioning a production database or object store.
- The client portal and hired-driver URL are workflow demonstrations only; URL possession must not be treated as authorisation.

13. Recommended Next Steps

Priority	Enhancement	Reason
P0	Add server-side authentication and RBAC	Protect trip, POD and billing data; separate driver, reviewer, billing and client access.
P0	Add central relational database and private object storage	Enable shared multi-user state, durable evidence and backup/restore.
P0	Replace query-string links with signed, expiring upload tokens	Prevent unauthorised access and replay of hired-driver uploads.
P0	Review AI provider terms and implement retention controls	Establish approved handling for real POD documents and personal data.
P1	Integrate SMS provider and delivery callbacks	Close the hired-driver notification loop.
P1	Implement finance-approved Tally export/API workflow	Reduce manual billing work and prevent reconciliation errors.
P1	Add audit logs, monitoring, alerts and operational metrics	Support traceability, reliability and management reporting.
P2	Pilot with a small branch and synthetic/training data	Validate usability, language support, process fit and measurable cycle-time improvement.

14. Conclusion

ProofFlow demonstrates a focused POD-to-billing workflow: capture delivery evidence, assist document review with AI, route exceptions to people, and make approved PODs visible to billing. The prototype makes the operational opportunity tangible while clearly identifying the work required for a secure, shared, production deployment.

Submission links

GitHub: ProofFlow repository
Live demo: ProofFlow production URL